

03 — Solution Explanation & End-to-End Technical Pipeline

End-to-End Data & Execution Flow Architecture

The Maritime Port Intelligence Platform (MPIP) processes data through an 8-stage transformation pipeline—from raw CSV records to interactive dashboard insights.

1. Data Sources Actual historical monthly port records (``data/port_cargo_monthly.csv``)

2. Data Ingest Pandas ingestion engine & custom upload route (``backend/app/routes/upload.py``)

3. Processing Feature engineering, commodity grouping & NMPA mapping (``forecasting.py``)

4. Backend FastAPI microservice application & route dispatcher (``backend/app/main.py``)

5. AI / ML Holt-Winters + Ridge Regression, Backtesting WAPE & Simulator (``forecasting.py``)

6. API Layer JSON REST Endpoints (``/cargo/forecast``, ``/trade/lanes``, ``/anomaly/events``, etc.)

7. Dashboard Streamlit UI layout, custom CSS cards & Plotly charts (``frontend/app.py``)

8. Port Insights Proactive berth allocation, risk mitigation & cargo decision support

Detailed Breakdown of Every Pipeline Stage

Stage 1: Data Sources

- **What Happens?:** Historical multi-year port cargo statistics and AIS vessel parameters are stored as persistent CSV files.
- **Which File Performs It?:** `CODE_PH_0` and `CODE_PH_1`.
- **Technology Used:** Standard CSV text formatting / File I/O.
- **Input Received:** Raw monthly port records (2021, 2023, 2024, 2025, 2026).
- **Output Produced:** 1,350+ structured raw dataset rows containing `CODE_PH_0`, `CODE_PH_1`, `CODE_PH_2` (LOADED/UNLOADED), `CODE_PH_3`, `CODE_PH_4`, `CODE_PH_5`, and YoY variations.

Stage 2: Data Ingestion

- **What Happens?:** Reads CSV files from disk or accepts custom CSV uploads via multipart HTTP form POST data. Validates column headers and missing fields.
- **Which File Performs It?:** `CODE_PH_0` (via Pandas `CODE_PH_1`) and `CODE_PH_2` (`CODE_PH_3`).
- **Technology Used:** Python Pandas, FastAPI `CODE_PH_0`, Python `CODE_PH_1`.
- **Input Received:** Disk file path or HTTP Multipart form binary stream.
- **Output Produced:** Pandas DataFrame in memory (`CODE_PH_0`) cleaned of corrupted entries.

Stage 3: Data Cleaning & Preprocessing

- **What Happens?:** Aggregates monthly tonnage by commodity and section (**CODE_PH_0/CODE_PH_1**), constructs continuous datetime indices (**CODE_PH_2**), computes rolling stats, YoY variations, and maps each commodity to NMPA facility definitions (Oil Jetties, JSW Container Terminal, UPCL Coal Berths, KIOCL Jetty).
 - **Which File Performs It?:** **CODE_PH_0** (function **CODE_PH_1**).
 - **Technology Used:** Pandas time-series resamplers, NumPy array transformers.
 - **Input Received:** Raw Pandas DataFrame.
 - **Output Produced:** Preprocessed time-series DataFrame with columns **CODE_PH_0** (date), **CODE_PH_1** (tonnes), **CODE_PH_2**, **CODE_PH_3**, **CODE_PH_4**, **CODE_PH_5**.
-

Stage 4: Backend Service Layer

- **What Happens?:** FastAPI orchestrates application startup, handles CORS headers, routes incoming REST requests, and dispatches queries to analytical service modules.
 - **Which File Performs It?:** **CODE_PH_0** and router modules in **CODE_PH_1** (**CODE_PH_2**, **CODE_PH_3**, **CODE_PH_4**, **CODE_PH_5**, **CODE_PH_6**, **CODE_PH_7**, **CODE_PH_8**).
 - **Technology Used:** FastAPI, Starlette, Pydantic, Uvicorn ASGI Server.
 - **Input Received:** HTTP GET/POST requests with query parameters (e.g., **CODE_PH_0**, **CODE_PH_1**, **CODE_PH_2**).
 - **Output Produced:** Structured JSON API response payloads containing calculation results, chart JSON arrays, and status codes.
-

Stage 5: Analytics & AI/ML Execution

- **What Happens?:** Executes the dual ML model pipeline:
 1. **Baseline Model:** Computes Seasonal Naive / 12-month rolling historical averages.
 2. **Primary ML Model:** Fits Holt-Winters Exponential Smoothing + Multi-Feature Ridge Regression with L2 regularization.
 3. **Confidence Bounds:** Calculates 95% dynamic prediction intervals using residual standard error scaling.
 4. **Model Backtesting:** Evaluates out-of-sample test performance (WAPE, MAPE, MAE, RMSE).
 5. **Explainability:** Calculates percentage driver weights for monsoon, hinterland, and market factors.
 6. **What-If Simulation:** Computes capacity stress level and simulated total throughput when sliders shift.
 - **Which File Performs It?:** **CODE_PH_0** (functions **CODE_PH_1**, **CODE_PH_2**, **CODE_PH_3**).
 - **Technology Used:** NumPy mathematical linear algebra, Pandas, Exponential Smoothing algorithms, Ridge Regression math.
 - **Input Received:** Query parameters (**CODE_PH_0**, **CODE_PH_1**, **CODE_PH_2**) or scenario payloads (**CODE_PH_3**, **CODE_PH_4**, **CODE_PH_5**).
 - **Output Produced:** Python dictionaries containing forecast values, upper/lower interval arrays, driver lists, WAPE error scores, and simulation summaries.
-

Stage 6: REST API Layer

- **What Happens?:** Serializes Python data structures into standardized JSON output formats and serves them over HTTP port 8000.
 - **Which File Performs It?:** *CODE_PH_0* (endpoints *CODE_PH_1*, *CODE_PH_2*, *CODE_PH_3*, *CODE_PH_4*, *CODE_PH_5*).
 - **Technology Used:** FastAPI JSONResponse / Pydantic serialization models.
 - **Input Received:** HTTP request endpoints.
 - **Output Produced:** Verified JSON payloads consumed by frontend clients.
-

Stage 7: Frontend Dashboard Rendering

- **What Happens?:** Fetches API data via HTTP requests, applies custom CSS styling, renders KPI metric cards, constructs Plotly interactive time-series graphs, and builds filter control dropdowns across 9 navigation tabs.
 - **Which File Performs It?:** *CODE_PH_0*.
 - **Technology Used:** Streamlit, Plotly Express, Plotly Graph Objects, HTML/CSS injects.
 - **Input Received:** JSON responses from backend REST endpoints (*CODE_PH_0*).
 - **Output Produced:** HTML5/JavaScript interactive dashboard UI rendered in the user's web browser (*CODE_PH_0*).
-

Stage 8: Port & Cargo Insights (User Decision Support)

- **What Happens?:** Port operations managers, customs officers, and harbor masters view forward-looking cargo throughput forecasts, evaluate risk warnings, run What-If simulations, and make informed operational decisions.
- **Which File Performs It?:** User web browser UI.
- **Technology Used:** Browser DOM / Interactive UI controls.
- **Input Received:** Rendered dashboard panels, Plotly tooltips, scenario sliders.
- **Output Produced:** Proactive berth draft preparation, optimized tugboat scheduling, streamlined rail freight staging, and zero vessel demurrage penalties.